EXPERIMENT - 58

1. Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k ≤ size. Pass the array to a function named kth_smallest() using its address. Use pointers to pass the array between the calling function and kth_smallest(). The function kth_smallest() must:

- Arrange the elements of arr[] in ascending order using the bubble sort algorithm.
- Find the element that occupies the kth position in the sorted array.
- Display the kth smallest element.
- Consider duplicate elements as separate elements.
- Display -1 if k is invalid.

For 7 10 4 3 20 15 with k = 3 the answer is 7, and with k = 4 it is 10. With k = 7 the answer is -1. For 4 2 2 7 5 with k = 3 the answer is 4, because the sorted array is 2 2 4 5 7 and the duplicates are counted separately.

2. Algorithm

Main Program

Step 1: START

Step 2: Read n and declare the variable length array arr of size n.

Step 3: Read the n elements of arr, then read k.

Step 4: Call kth_smallest(&arr[0], n, k).

Step 5: STOP

Function kth_smallest()

Step 1: If k is less than or equal to 0, or k is greater than n, display −1 and return to the calling function.

Step 2: Repeat for i = 0 to n − 2:

Repeat for j = 0 to n − 2 − i:

If arr[j] is greater than arr[j + 1], swap arr[j] and arr[j + 1] using temp.

Step 3: Display arr[k − 1].

Step 4: Return to the calling function.

3. Test Case Table

#	Array	k	Expected Output	Actual Output	Result
1	7 10 4 3 20 15	3	7, the third of 3 4 7 10 15 20	Matches example 1 in the question	**PASS**
2	7 10 4 3 20 15	4	10	Matches example 2	**PASS**
3	7 10 4 3 20 15	7	−1, as k is bigger than the size	Matches example 3	**PASS**

#	Array	k	Expected Output	Actual Output	Result
4	4 2 2 7 5	3	4, the third of 2 2 4 5 7	Matches example 4	**PASS**
5	4 2 2 7 5	1	2, the smallest value	2	**PASS**
6	4 2 2 7 5	5	7, the largest value	7, so k = n is still valid	**PASS**
7	4 2 2 7 5	0	−1, as k must be above 0	−1	**PASS**
8	4 2 2 7 5	2	2 again, since the second copy counts on its own	2, the duplicate was not skipped	**PASS**

4. Source Code

kthsmallest58.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

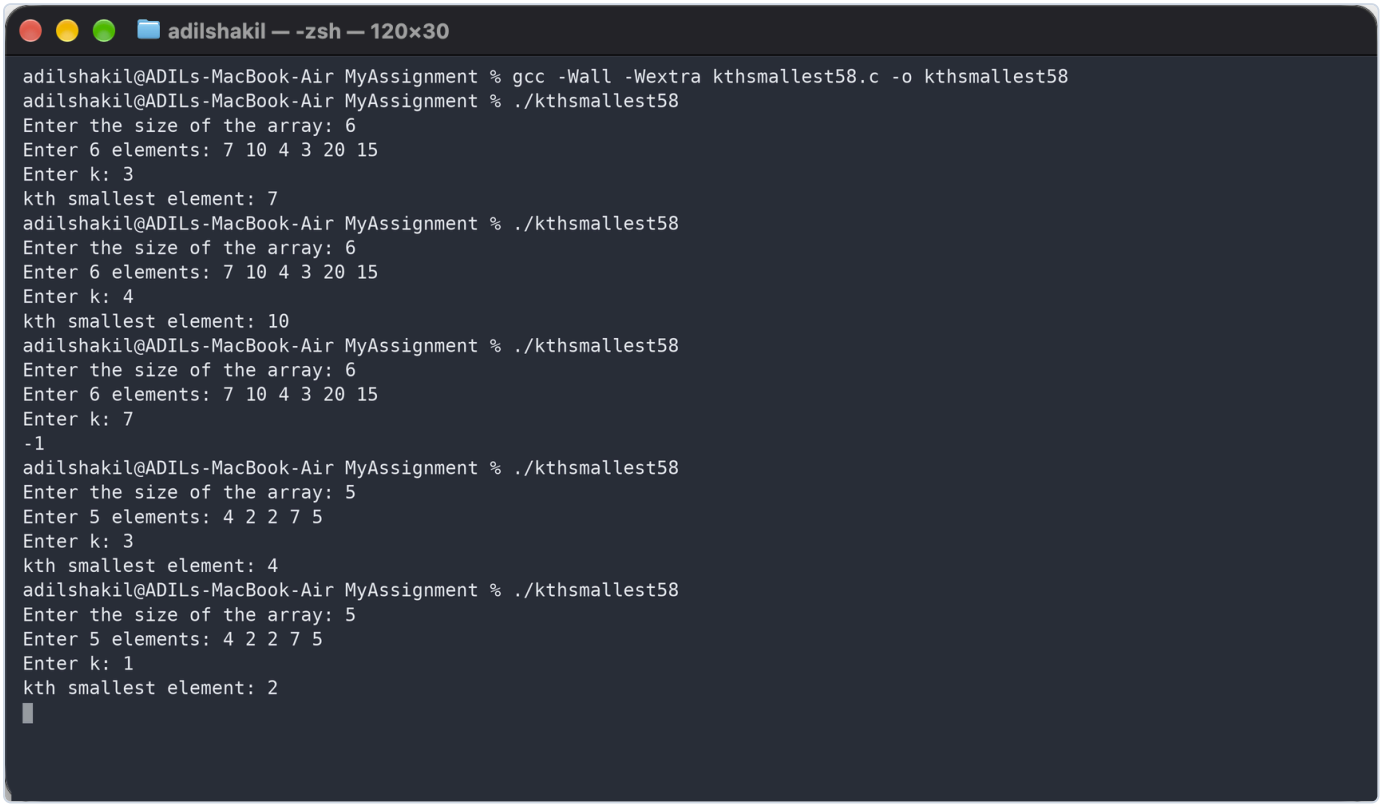
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

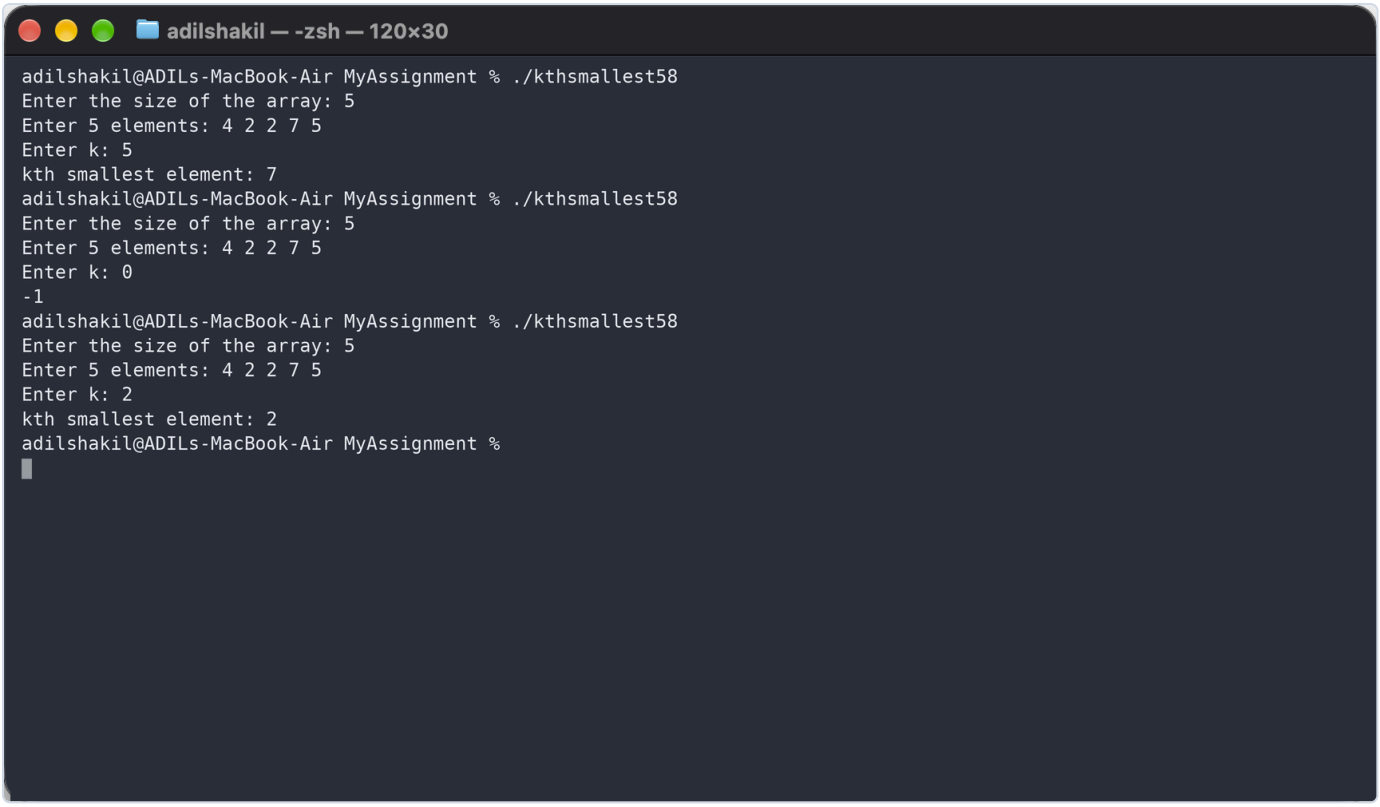
5. Compilation and Output

Compiled with `gcc -Wall -Wextra kthsmallest58.c -o kthsmallest58` so the small warnings would show up too. It built clean.

The Four Examples from the Question, and k = 1



k at the Top End, an Invalid k, and a Duplicate



All eight cases came out right, and the first four are the examples given in the question.

kth_smallest() takes `int *arr`, so it is handed the address of the array and not a copy of it, and the call is written as `&arr[0]`.

The one that actually caught me is `arr[k - 1]`. Counting in C starts at 0, so the 3rd element sits at index 2. I wrote `arr[k]` first and every answer came out one place too far along, which for case 1 meant 10 instead of 7.

The check on k is done before the sort rather than after it. If k is outside 1 to n there is no kth element to find, so sorting first would only be wasted work.

Counting duplicates separately needs nothing extra. Bubble sort only moves values around and never removes any, so in 4 2 2 7 5 both copies of 2 keep their own place in 2 2 4 5 7. Case 8 is the test for that: k = 2 gives 2 and not 4.

Cases 5, 6 and 7 I added myself, for k at the bottom, k at the top and a k below the allowed range. Case 6 matters because k = n is still valid, so the test has to be `k > n` and not `k >= n`.